Tutorial No : 8

Tutorial Title: Implement Binary search tree with operations Create, search, and recursive traversal.

Name of Student: Shweta Lohakare

Class: SY **Div:** 2 **PRN No:** B25ET1099 **Batch:** B2

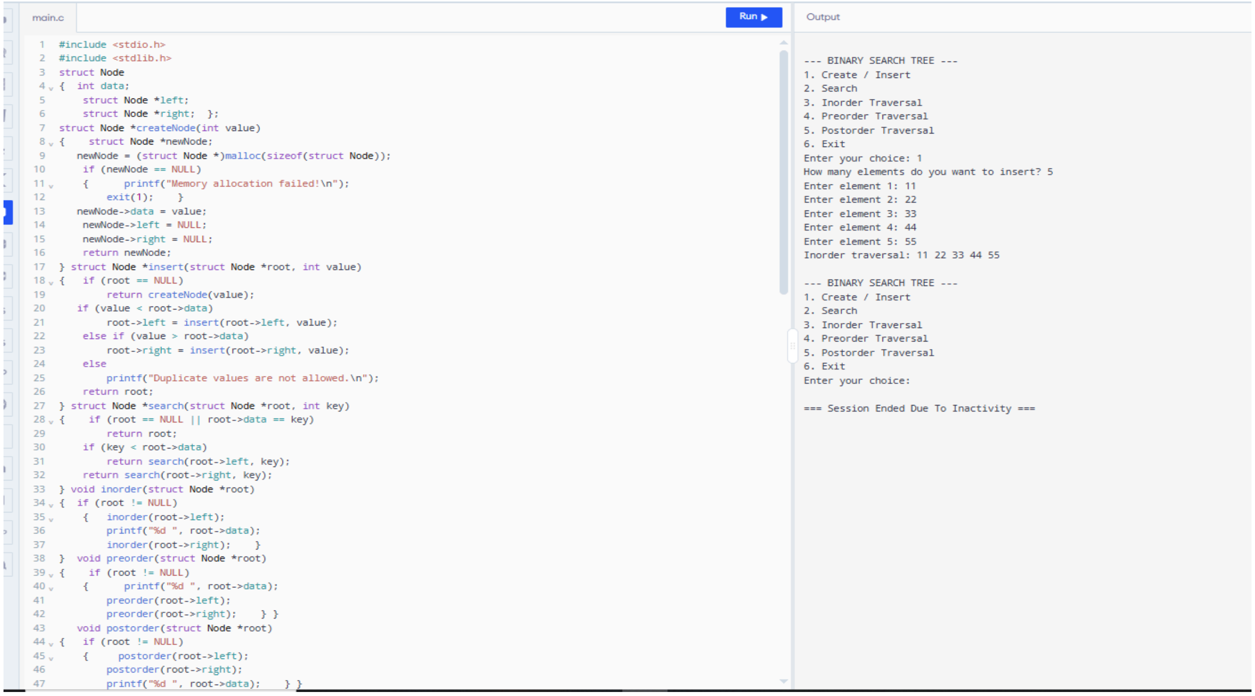
Program:

```
#include <stdio.h>
#include <stdlib.h>
struct Node
{  int data;
   struct Node *left;
   struct Node *right;  };
struct Node *createNode(int value)
{    struct Node *newNode;
  newNode = (struct Node *)malloc(sizeof(struct Node));
   if (newNode == NULL)
   {      printf("Memory allocation failed!\n");
       exit(1);    }
  newNode->data = value;
   newNode->left = NULL;
   newNode->right = NULL;
   return newNode;
} struct Node *insert(struct Node *root, int value)
{   if (root == NULL)
       return createNode(value);
  if (value < root->data)
       root->left = insert(root->left, value);
   else if (value > root->data)
       root->right = insert(root->right, value);
   else
       printf("Duplicate values are not allowed.\n");
   return root;
} struct Node *search(struct Node *root, int key)
{    if (root == NULL || root->data == key)
       return root;
   if (key < root->data)
       return search(root->left, key);
```

```
    return search(root->right, key);
} void inorder(struct Node *root)
{  if (root != NULL)
    {   inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);    }
}  void preorder(struct Node *root)
{    if (root != NULL)
    {      printf("%d ", root->data);
        preorder(root->left);
        preorder(root->right);    } }
  void postorder(struct Node *root)
{   if (root != NULL)
    {     postorder(root->left);
        postorder(root->right);
        printf("%d ", root->data);    } }
void freeTree(struct Node *root)
{    if (root != NULL)
    {   freeTree(root->left);
        freeTree(root->right);
        free(root);    } }
int main()
{  struct Node *root = NULL;
    struct Node *result;
    int choice, value, n, i;
  do
    {      printf("\n--- BINARY SEARCH TREE ---\n");
        printf("1. Create / Insert\n");
        printf("2. Search\n");
        printf("3. Inorder Traversal\n");
        printf("4. Preorder Traversal\n");
        printf("5. Postorder Traversal\n");
        printf("6. Exit\n");
        printf("Enter your choice: ");
    if (scanf("%d", &choice) != 1)
            break;
     switch (choice)
        {    case 1:
                printf("How many elements do you want to insert? ");
            if (scanf("%d", &n) != 1 || n < 0)
```

```
                {    printf("Invalid input!\n");
                    choice = 6;
                    break;     }
              for (i = 0; i < n; i++)
                {        printf("Enter element %d: ", i + 1);
                      if (scanf("%d", &value) != 1)
                    {          choice = 6;
                 break;
                    }        root = insert(root, value);
                }   printf("Inorder traversal: ");
                inorder(root);
                printf("\n");
                break;
       case 2:
                printf("Enter element to search: ");
               if (scanf("%d", &value) != 1)
                {       choice = 6;
                    break;         }
          result = search(root, value);
                if (result != NULL)
                    printf("%d found in the tree.\n", value);
                else
                    printf("%d not found in the tree.\n", value);
         break;
        case 3:
            printf("Inorder Traversal: ");
                inorder(root);
                printf("\n");
                break;
                   case 4:
                printf("Preorder Traversal: ");
                preorder(root);
                printf("\n");
                break;
             case 5:
                printf("Postorder Traversal: ");
                postorder(root);
                printf("\n");
                break;
         case 6:
                printf("Exiting program.\n");
                break;
        default:
                printf("Invalid choice!\n");        }
    } while (choice != 6);
   freeTree(root);
    return 0;
}
```

Output:



```
#include <stdio.h>
#include <stdlib.h>
struct Node
{  int data;
    struct Node *left;
    struct Node *right;  };
struct Node *createNode(int value)
{    struct Node *newNode;
   newNode = (struct Node *)malloc(sizeof(struct Node));
    if (newNode == NULL)
    {      printf("Memory allocation failed!\n");
        exit(1);    }
   newNode->data = value;
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;
} struct Node *insert(struct Node *root, int value)
{   if (root == NULL)
        return createNode(value);
   if (value < root->data)
        root->left = insert(root->left, value);
    else if (value > root->data)
        root->right = insert(root->right, value);
    else
        printf("Duplicate values are not allowed.\n");
    return root;
} struct Node *search(struct Node *root, int key)
{    if (root == NULL || root->data == key)
        return root;
    if (key < root->data)
        return search(root->left, key);
    return search(root->right, key);
} void inorder(struct Node *root)
{  if (root != NULL)
    {   inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);    }
}  void preorder(struct Node *root)
{    if (root != NULL)
    {      printf("%d ", root->data);
        preorder(root->left);
        preorder(root->right);    } }
   void postorder(struct Node *root)
{   if (root != NULL)
    {     postorder(root->left);
        postorder(root->right);
        printf("%d ", root->data);    } }
```

```
--- BINARY SEARCH TREE ---
1. Create / Insert
2. Search
3. Inorder Traversal
4. Preorder Traversal
5. Postorder Traversal
6. Exit
Enter your choice: 1
How many elements do you want to insert? 5
Enter element 1: 11
Enter element 2: 22
Enter element 3: 33
Enter element 4: 44
Enter element 5: 55
Inorder traversal: 11 22 33 44 55

--- BINARY SEARCH TREE ---
1. Create / Insert
2. Search
3. Inorder Traversal
4. Preorder Traversal
5. Postorder Traversal
6. Exit
Enter your choice:

=== Session Ended Due To Inactivity ===
```